'h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-2076f10863aad887aeeac8dc56e8.html

求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

第 26 章：`contextvars`、日志与异步装饰器

先记住一句话

`ContextVar` 让同一个变量在不同异步执行上下文中拥有不同的绑定，适合保存 `request_id`、`run_id` 这类贯穿调用链的诊断信息；它不是普通全局变量，也不能简单等同于 Java 的 `ThreadLocal`。

一个 Agent 请求通常会同时调用 LLM、HTTP、数据库和多个 tool。多个请求又可能在同一个事件循环线程中交错运行。如果日志只写“tool 开始”和“tool 完成”，却不知道它们属于哪个请求，故障排查几乎只能靠猜。

本章要建立的是下面这条完整数据流：

```
请求 Task 设置 request_id / run_id
             |
             ├─ 创建 tool Task：复制创建点的 Context
             |
             ├─ tool wrapper：临时设置 tool_name，并包住真正的 await
             |
             └─ logging.Filter：读取当前 Context，把字段写入 LogRecord

请求结束：finally 按相反顺序 reset，恢复进入请求前的 Context
```

学完后，你应该能解释：

- 为什么单线程事件循环中的普通全局 `request_id` 仍会串值；
- `ContextVar.get()`、`set()`、`Token`、`reset()` 各自负责什么；
- 子 Task 继承的是哪个时刻的 Context，父子后续修改为何互不回写；
- 为什么复制 Context 不等于深拷贝其中的可变对象；
- `asyncio.to_thread()` 在本章场景中何时取得 Context，为什么不能推广到所有线程入口；
- 什么适合放入隐式上下文，什么仍应通过参数或依赖对象明确传递；
- 怎样让日志和计时覆盖协程真正执行与等待 I/O 的全过程；
- 为什么异常和取消必须继续传播，async generator 又为什么需要单独的 wrapper。

1. Java → Python 心智模型

Java 中熟悉的概念	Python 中可以先想到	不能画等号的地方
`static` 可变字段	模块级全局变量	两者都由并发调用共享，都不能隔离请求状态
SLF4J MDC / `ThreadLocal` 中的关联 ID	`ContextVar`	MDC 常以物理线程为边界；`ContextVar` 的绑定属于当前 `Context`，`asyncio.Task` 原生支持复制与恢复
提交任务前复制 MDC	`asyncio.create_task()` 在创建点复制 Context	Python 只复制 Context 中的绑定映射，不深拷贝绑定的对象
`try/finally` 恢复旧 MDC	`token = var.set(...)` 后在 `finally` 中 `var.reset(token)`	Token 记录精确的旧绑定，不能靠写死默认值代替
日志框架的 MDC 字段注入	`logging.Filter` 为 `LogRecord` 补字段	Python 标准库不自动规定字段、脱敏或跨进程传播，需要应用自己配置
AOP `@Around` 拦截器	返回 wrapper 的函数装饰器	Python 装饰器在定义时替换函数对象；堆叠顺序会改变注册表最终保存哪个对象
`CompletableFuture` 的执行阶段计时	`async def wrapper` 中 `await` 原调用	同步 wrapper 调用异步函数时只得到 coroutine，对业务函数体尚未开始执行
任务取消并向上报告	`asyncio.CancelledError` 传播	Python 取消是协作式的；捕获后通常应清理并重新抛出，不能当普通成功结果返回

最容易误判的是 `ThreadLocal` 类比。可以先把 `ContextVar` 理解成“异步代码可用的 MDC 数据来源”，但不能完全等同：

- `ThreadLocal` 绑定在工作线程上。线程池复用、响应式流切线程或 `CompletableFuture` 换 executor 时，通常需要框架或 wrapper 显式搬运 MDC。
- `ContextVar` 的值绑定在 `contextvars.Context` 上。`asyncio` 在创建 Task 时会复制当前 Context，因此多个 Task 即使运行在同一线程，也能读取各自的绑定。
- Python 的自动传播只覆盖 API 明确承诺的边界。它不会自动穿过任意线程池、子进程、消息队列或 HTTP 调用。跨服务仍要使用 trace header、消息元数据等显式协议。

所以真正要记住的不是“`ContextVar` 就是 Python 版 `ThreadLocal`”，而是：**先确认当前代码跨越**

README
README
lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-2076f10863aad887aeeac8dc56e8.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

所以真正要记住的不是“`ContextVar` 就是 Python 版 `ThreadLocal`”，而是：**先确认当前代码跨越的是调用栈、Task、线程、进程还是网络，再确认那个边界的传播规则。**

2. 为什么普通全局变量一定会串请求

事件循环线程在同一瞬间通常只执行一个 Task 的 Python 代码，但 Task 会在 `await` 处暂停，让其他 Task 继续运行：

```
请求 A: GLOBAL_REQUEST_ID = "REQ-A" ── await I/O ──────────→ 再读取
                                          │
请求 B:                GLOBAL_REQUEST_ID = "REQ-B"
```

模块中只有一个 `GLOBAL_REQUEST_ID` 绑定。请求 A 暂停时，请求 B 把它改成了 `REQ-B`；A 恢复后读取的仍是模块当前值，不会神奇地“回到 A 自己的副本”。

这不是只有多线程才会出现的数据竞争，而是多个并发控制流共享可变状态产生的逻辑串扰。下面的最小例子不依赖线程：

```python
import asyncio

CURRENT_REQUEST = "-"


async def handle(request_id: str) -> str:
    global CURRENT_REQUEST
    CURRENT_REQUEST = request_id
    await asyncio.sleep(0)  # 当前 Task 可能在这里让出执行权
    return CURRENT_REQUEST
```

并发调用 `handle("A")` 和 `handle("B")` 时，两个结果可能都读到 B，也可能随调度顺序变化。单次测试“碰巧正确”不能证明设计安全。

用 Lock 保护一次赋值也解决不了问题：锁释放后，别的请求仍能在本请求下次读取前改值。若把整个请求生命周期都锁住，虽然不串值，却把所有请求强制串行，失去了异步并发的意义。

问题需要的不是“更小心地共享同一个值”，而是“让不同执行上下文拥有不同绑定”。

3. `ContextVar` 的生命周期：设置、使用、精确恢复

README
README
f7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-2076f10863aad887aeeac8dc56e8.html
关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

3. `ContextVar` 的生命周期：设置、使用、精确恢复

3.1 在模块顶层声明

```
from contextvars import ContextVar

REQUEST_ID = ContextVar[str]("request_id", default="-")
RUN_ID = ContextVar[str]("run_id", default="-")
TOOL_NAME = ContextVar[str]("tool_name", default="-")
```

- `ContextVar[str]` 告诉静态检查器这里约定保存字符串；
- `"request_id"` 是变量的诊断名称；
- `default="-"` 表示当前 Context 没有显式绑定时，`get()` 返回 `"-"`；
- `REQUEST_ID` 变量对象可以全局共享，随 Context 变化的是“该对象当前绑定了什么值”。

官方建议在模块顶层创建 `ContextVar`。`Context` 会持有变量的强引用，不应为每次请求在闭包中反复创建新变量。参见 Python 3.14 `ContextVar`。

没有声明默认值的变量，在当前 Context 也没有绑定时，`get()` 会抛 `LookupError`。两种策略都合理：日志字段常使用安全占位符；必须由入口设置的安全上下文，则可以故意不提供默认值，让遗漏尽早暴露。

3.2 `set()` 返回的 Token 是恢复凭证

```
token = REQUEST_ID.set("REQ-100")
try:
    await handle_request()
finally:
    REQUEST_ID.reset(token)
```

`set()` 不是修改一个进程级字段，而是在当前 Context 中建立新绑定，并返回一个 `Token`。Token 记住本次设置之前的精确状态，`reset(token)` 因而能恢复外层。

不要用下面的代码代替：

```
# 反例：外层也许不是默认值，而是另一个真实请求或调用上下文。
finally:
    REQUEST_ID.set("-")
```

```
# 反例：外层也许不是默认值，而是另一个真实请求或调用上下文。
finally:
    REQUEST_ID.set("-")
```

假设外层已经绑定 `REQ-OUTER`，内层临时绑定 `REQ-INNER`。写死 `"-"` 会破坏外层；用内层 Token reset 才会恢复 `REQ-OUTER`。这与 Java 代码在 `finally` 中恢复进入方法前的 MDC 值是同一类生命周期问题，但 Token 已经替你保存了"旧值"和"此前根本没有显式绑定"的差别。

3.3 嵌套绑定按栈的反方向退出

```
outer = REQUEST_ID.set("REQ-OUTER")
try:
    inner = REQUEST_ID.set("REQ-INNER")
    try:
        assert REQUEST_ID.get() == "REQ-INNER"
    finally:
        REQUEST_ID.reset(inner)

    assert REQUEST_ID.get() == "REQ-OUTER"
finally:
    REQUEST_ID.reset(outer)
```

后 `set()` 的绑定先 `reset()`。一个 Token 只属于创建它的变量、Context 和那一次设置，不能重复 reset，也不能拿去恢复另一个 `ContextVar`。

`finally` 覆盖正常返回、普通异常和取消，所以请求入口、middleware、tool wrapper 都应该把恢复动作放在这里。

Python 3.14 起，Token 也可以作为同步上下文管理器：

```
with REQUEST_ID.set("REQ-100"):
    await handle_request()
```

离开 `with` 时会自动恢复。它只是 `set/try/finally/reset` 的简写，不是另一套隔离机制；阅读旧版本代码和排查嵌套问题时，仍需理解完整生命周期。

4. Task 复制 Context 的时机

4. Task 复制 Context 的时机

`asyncio.create_task(coro)` 默认会在调用 `create_task` 的时刻复制当前 Context，让新 Task 在副本中运行。不是等子协程第一次真正执行时才读取父 Task 的最新值。

```
REQUEST_ID.set("REQ-BEFORE-CREATE")
child = asyncio.create_task(child_work())
REQUEST_ID.set("REQ-AFTER-CREATE")
```

结果是：

- 子 Task 开始时读取 `REQ-BEFORE-CREATE`；
- 父 Task 后来的 `set()` 只改变父 Task 当前 Context；
- 子 Task 自己再 `set()`，也不会反向更新父 Task；
- 父子若要交换业务结果，应使用返回值、Task 结果、Queue 或明确共享状态，不能期待 `ContextVar` 回写。

这和在线程池提交前拍一份 MDC 快照有相似之处，但 asyncio 的复制是 Task API 的原生语义。官方合同见 Python 3.14 `create_task()`。

请求入口的正确顺序通常是：

```
request_token = REQUEST_ID.set(incoming_request_id)
try:
    llm_task = asyncio.create_task(call_llm())
    db_task = asyncio.create_task(call_database())
    return await asyncio.gather(llm_task, db_task)
finally:
    REQUEST_ID.reset(request_token)
```

若先创建 Task，再设置请求 Context，已创建的 Task 不会追溯继承新值。`TaskGroup.create_task()` 也应遵守同样的创建点思维。高级代码可以通过 `context=` 显式指定 Context，但普通请求处理通常不需要这样做。

Context 副本不是对象深拷贝

复制的是 `ContextVar -> value` 这张绑定映射，不是递归复制 value 指向的 Python 对象：

Context 副本不是对象深拷贝

复制的是 `ContextVar -> value` 这张绑定映射，不是递归复制 value 指向的 Python 对象：

```
EVENTS = ContextVar[dict[str, list[str]]]("events")
bag = {"items": []}
EVENTS.set(bag)

first = asyncio.create_task(child_a())
second = asyncio.create_task(child_b())
```

两个子 Task 的 Context 都可能把 `EVENTS` 绑定到同一个 `bag` 。如果一个子 Task 执行：

```
EVENTS.get()["items"].append("changed")
```

它原地修改的是共享 dict 中的共享 list，并没有改变 ContextVar 绑定。父 Task 和兄弟 Task 都可能看到变化。`copy_context()` 也不会替你做 `deepcopy` 。

因此 ContextVar 最适合保存短小、不可变的值，例如 request ID、run ID、租户诊断标签。若必须更新复合值，应构造新对象后重新 `set()` ；若本来就需要共享可变状态，则明确设计所有权、同步和消息传递，不要借 ContextVar 把它隐藏起来。

5. 跨线程时先看具体 API 的合同

5.1 `asyncio.to_thread()` 会传播当前 Context

`asyncio.to_thread(func, *args)` 会在线程中执行同步 callable，并传播当前 `contextvars.Context` 。但要同时记住：调用 `to_thread()` 只创建 coroutine，它的函数体要在 coroutine 真正开始运行时才执行。

本章在 CPython 3.14.7 中验证了两种不同时间线。

只创建 coroutine，改值后再直接 await：

```
REQUEST_ID.set("REQ-WHEN-CALLED")
deferred = asyncio.to_thread(read_request_id)

REQUEST_ID.set("REQ-WHEN-AWAITED")
```

README
README
./7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-2076f10863aad887aeeac8dc56e8.html
需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

```
REQUEST_ID.set("REQ-WHEN-CALLED")
deferred = asyncio.to_thread(read_request_id)

REQUEST_ID.set("REQ-WHEN-AWAITED")
result = await deferred
```

`to_thread` 在直接 await 时才开始运行并复制 Context，所以线程读到 `REQ-WHEN-AWAITED`。

先把 `to_thread` coroutine 创建成 Task：

```
REQUEST_ID.set("REQ-WHEN-TASK-CREATED")
task = asyncio.create_task(asyncio.to_thread(read_request_id))

REQUEST_ID.set("REQ-PARENT-CHANGED-LATER")
result = await task
```

这里先发生 Task 创建点的 Context 复制。`to_thread` 以后在子 Task 的 Context 中开始，再把那份 Context 传播给工作线程，所以线程读到 `REQ-WHEN-TASK-CREATED`。

```
父 Task 当前 Context
   | create_task 时复制
   ▼
承载 to_thread 的子 Task Context
   | to_thread 开始运行时复制，并在线程中 Context.run
   ▼
工作线程中的同步 callable
```

公开语义见 `asyncio.to_thread()`，
复制位置可参照 CPython 3.14 `Lib/asyncio/threads.py`。

5.2 `run_in_executor()` 不会自动提供同样保证

不要把 `to_thread` 的规则推广成“Python 线程池都会继承 Context”。本章的新单线程池实验中：

```
run_in_executor 直接提交，线程看到：-
显式 Context.run 提交，线程看到：REQ-EXECUTOR-CALLER
```

README
README
s/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-2076f10863aad887aeeac8dc56e8.html
需求相关...
eclp相关
配运运单26年需求...
运单小秘值班安排
研发日常任务
运单运维
运单-资源大屏
20260618-应急预案
运单26年需求迭代...
商家下单计数

5.2 `run_in_executor()` 不会自动提供同样保证

不要把 `to_thread` 的规则推广成"Python 线程池都会继承 Context"。本章的新单线程池实验中：

```
run_in_executor 直接提交，线程看到：-
显式 Context.run 提交，线程看到：REQ-EXECUTOR-CALLER
```

需要显式传播时，可以在提交前复制：

```
context = contextvars.copy_context()
result = await loop.run_in_executor(executor, context.run, sync_function)
```

这很像 Java executor 中显式包装 Runnable/Callable 来复制和恢复 MDC。每次并发提交都应获得自己的 Context 副本；不要让同一个已进入的 Context 同时在多个线程运行。

`to_thread` 也只负责适配同步调用和传播 Context。取消等待它的 Task，并不会强行终止已经运行的线程。底层 timeout、副作用、线程安全和 executor 关闭责任仍要单独设计。

6. 哪些信息适合隐式传递

ContextVar 能避免把横切元数据穿过每一层函数签名，但"可以隐式传"不等于"都应该藏起来"。

信息	推荐方式	判断依据
`request_id` 、 `run_id` 、trace/span 关联 ID	ContextVar	用于诊断关联，不决定核心业务结果
当前租户的日志标签	谨慎使用 ContextVar	可辅助诊断； 真正授权判断仍应经过清晰、 可测试的安全边界
用户问题、tool 输入、SQL 参数	显式参数	它们决定函数做什么， 应该从签名和调用点可见
HTTP client、数据库 session、repository	显式参数或有生命周期的对象字段	它们有容量、事务、关闭与测试替换责任
API Key、访问令牌	依赖边界显式管理，禁止写日志	放进 Context 不会让凭据自动安全， 反而可能扩大可见范围

README README
rs/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-2076f10863aad887aeeac8dc56e8.html
的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

		及而可能扩大可见范围
可变步骤结果、消息列表	返回值、Queue 或有所有者的状态	需要明确数据流；Context 的浅复制不能提供隔离

一个实用问题是：**去掉这个值后，函数只是少了诊断关联，还是连业务行为都无法决定？**前者可能适合 ContextVar，后者通常应该显式传参。

这也解释了为什么 ContextVar 不是依赖注入容器。把 client、session 和业务输入全塞进去，虽减少了参数，却隐藏了资源所有权和函数真正依赖，测试会比 Java 中滥用静态 Service Locator 同样困难。

7. 让每条日志在输出边界取得当前上下文

Python 标准库 `logging` 可以通过 `Filter` 在 `LogRecord` 进入 Handler 前补字段：

```
import asyncio
import logging
import threading


class ContextFilter(logging.Filter):
    def filter(self, record: logging.LogRecord) -> bool:
        record.request_id = REQUEST_ID.get() ^# type: ignore[attr-defined]
        record.run_id = RUN_ID.get()  # type: ignore[attr-defined]
        record.tool_name = TOOL_NAME.get()  # type: ignore[attr-defined]
        try:
            task = asyncio.current_task()
        except RuntimeError:  #. 当前工作线程没有事件循环
            task = None
        record.task_name = (  # type: ignore[attr-defined]
            task.get_name() if task else f"thread:{threading.current_thread().name}"
        )
        return True
```

Formatter 随后可以使用这些字段：

```
logging.Formatter(
    "%(levelname)s request=%(request_id)s run=%(run_id)s "
    "task=%(task_name)s tool=%(tool_name)s | %(message)s"
```

职责关系类似“日志框架在输出时从 MDC 读取字段”，但这里的 MDC 数据源是当前 Context。Filter 不负责建立请求上下文；请求入口仍要先 `set()`，结束时仍要 `reset()`。

本章完整示例还给 Task 显式命名，并通过 wrapper 临时设置 `tool_name`。一条工具日志的字段来源是：

字段	产生位置	隔离原因
`request_id`	请求入口	每个请求 Task 有自己的 Context，工具 Task 在创建点复制
`run_id`	请求入口	与 request_id 使用相同生命周期
`task_name`	`create_task(..., name=...)`	Filter 从当前 Task 读取，不依赖全局可变字段
`tool_name`	异步 wrapper 入口	每次工具调用在自己的 Context 中临时 set，并在 finally 恢复
`elapsed_ms`	wrapper 的局部变量	每次调用有独立栈帧，计时包住完整 await

输出类似：

```
INFO request=REQ-100 run=RUN-ALPHA task=context:REQ-100:lookup:1 \
tool=lookup_documents | 工具调用成功 status=success elapsed_ms=0.134
```

标准库的相关做法可参考 Logging Cookbook：Use of contextvars。

日志要能关联，也要克制

通常适合记录：

- 经过审查的关联 ID、工具名、模型名和依赖名；
- started、success、error、cancelled、retry 等有限状态；
- 状态码、有界计数、耗时和异常类型；
- 按环境与安全要求审查后的 traceback。

通常不要直接记录：

- 完整 prompt、用户文档、HTTP 请求或响应正文；
- `Authorization`、Cookie、API Key、数据库密码；
- 未脱敏的个人信息；
- 任意 `args`、`kwargs` 的 `repr()`；

ırs/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-2076f10863aad887aeeac8dc56e8.html
的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

通常不要直接记录：

- 完整 prompt、用户文档、HTTP 请求或响应正文；
- `Authorization`、Cookie、API Key、数据库密码；
- 未脱敏的个人信息；
- 任意 `args`、`kwargs` 的 `repr()`；
- 未审查的异常文本，因为 SDK 异常可能带 URL、正文或凭据。

日志平台、采样、trace 导出与脱敏管线不在本章范围内。这里解决的是字段从哪里来、并发时归属是否正确，以及成功、失败、取消有没有被如实记录。

8. 异步装饰器必须包住真正的 `await`

8.1 同步 wrapper 只测到 coroutine 创建

下面的代码看起来包住了调用，实际并没有包住异步执行：

```
def wrong(function):
    @wraps(function)
    def wrapper(*args, **kwargs):
        started = perf_counter_ns()
        coroutine = function(*args, **kwargs)
        report(perf_counter_ns() - started)
        return coroutine

    return wrapper
```

调用 `async def` 函数只创建 coroutine 对象。原函数体第一行还没有执行，网络、数据库和其他 I/O 更没有开始。这段 wrapper 测到的只是对象创建开销。

对应到 Java，可以类比为只测量“创建一个尚未启动的异步描述对象”，却没有测量 future 真正完成。异步耗时的终点必须和异步完成信号绑定。

8.2 wrapper 自己也要是 `async def`

本章使用 Python 3.14 支持的类型参数语法，保留原函数的参数关系和最终结果类型：

```
def observe_async[**P, R](
```

```python
def observe_async[**P, R](
    function: Callable[P, Awaitable[R]],
) -> Callable[P, Coroutine[Any, Any, R]]:
    @wraps(function)
    async def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        token = TOOL_NAME.set(function.__name__)
        started = perf_counter_ns()
        try:
            result = await function(*args, **kwargs)
        except asyncio.CancelledError:
            log_cancelled(perf_counter_ns() - started)
            raise
        except Exception:
            log_error(perf_counter_ns() - started)
            raise
        else:
            log_success(perf_counter_ns() - started)
            return result
        finally:
            TOOL_NAME.reset(token)

    return wrapper
```

计时起点在调用原函数之前，终点在 `await` 正常完成、抛异常或收到取消之后，所以测量的是这次调用的墙钟耗时，包含协程实际运行和等待 I/O 的时间。它不是 CPU 时间，也不应测试某次运行必须精确等于多少毫秒。

终态处理有三条硬规则：

1. success 放在 `try` 的 `else`，只有 `await` 真正返回才记录成功；
2. 普通异常可以记录，但随后裸 `raise`，调用方仍收到原异常；
3. `CancelledError` 单独记录为 cancelled，完成清理后也必须继续抛出。

`asyncio.CancelledError` 直接继承 `BaseException`。吞掉取消会让 timeout、TaskGroup 和请求中止误以为下游已经正常结束。官方建议见 Task cancellation。

README README
ers/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-2076f10863aad887aeeac8dc56e8.html
的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

8.3 `wraps` 和参数规格解决的是两件事

`@wraps(function)` 保留函数名、docstring、注解和 `__wrapped__` 链，`inspect.signature()` 通常因此仍展示原始签名。它解决运行时元信息问题。

`P` 是参数规格，记录位置参数、仅位置参数、关键字参数、仅关键字参数和默认值之间的关系；`P.args` 、 `P.kwargs` 把这份关系传给 wrapper。它解决静态检查问题。

两者不能互相替代。可以把 `wraps` 类比成保留 Java 反射可见的方法元数据，把参数规格类比成泛型地保持函数式接口的输入输出关系，但 Python 装饰器仍是一个新的 callable 对象。

还要区分两种"返回类型":

```
async def lookup(query: str) -> Document: ...
```

这里的 `-> Document` 表示 coroutine 完成后交付的结果。调用 `lookup("python")` 当下得到的是 coroutine，不是 `Document` 。

从装饰器接收"可调用对象"的角度描述，调用结果应写成:

```
Callable[P, Awaitable[R]]
```

本章 wrapper 本身由 `async def` 定义，调用后具体得到 `Coroutine[Any, Any, R]` 。不要把原异步函数定义误写成 `async def lookup(...) -> Awaitable[Document]` ；那表示 await 第一层后还会得到一个 Awaitable，多出了一层协议。

9. 装饰器顺序决定注册表保存谁

Python 装饰器从下往上应用:

```
@register_tool("lookup")
@observe_async
async def lookup_documents(...):
    ...
```

等价于:

```
lookup_documents = register_tool("lookup")(observe_async(lookup_documents))
```

等价于：

```
lookup_documents = register_tool("lookup")(observe_async(lookup_documents))
```

先创建观测 wrapper，再把 wrapper 注册。因此直接调用公开名字和从注册表动态调用，都会经过日志与计时层。

如果顺序反过来，注册器会先保存原函数，之后公开名字才被包装。从注册表取出的对象会绕过观测层。这与 Java 注解顺序本身不完全等价，更接近"哪个代理对象最终被容器登记"。判断时不要只看源码上有几个 `@`，要按从下到上的函数变换展开。

异构工具注册表是动态类型边界。字符串运行时才决定具体工具，普通 `dict` 无法让静态检查器从任意字符串恢复每个工具的精确签名。真实 Agent 还需要 tool schema、输入校验和结果校验；`ContextVar` 与装饰器不会替代这些业务边界。

10. async generator 不是"一次 await 得到最终结果"

本章 wrapper 的合同很窄：调用被包装对象得到 Awaitable，`await` 一次得到最终 `R`。

```
async def token_stream() -> AsyncGenerator[str]:
    yield "token-1"
    yield "token-2"
```

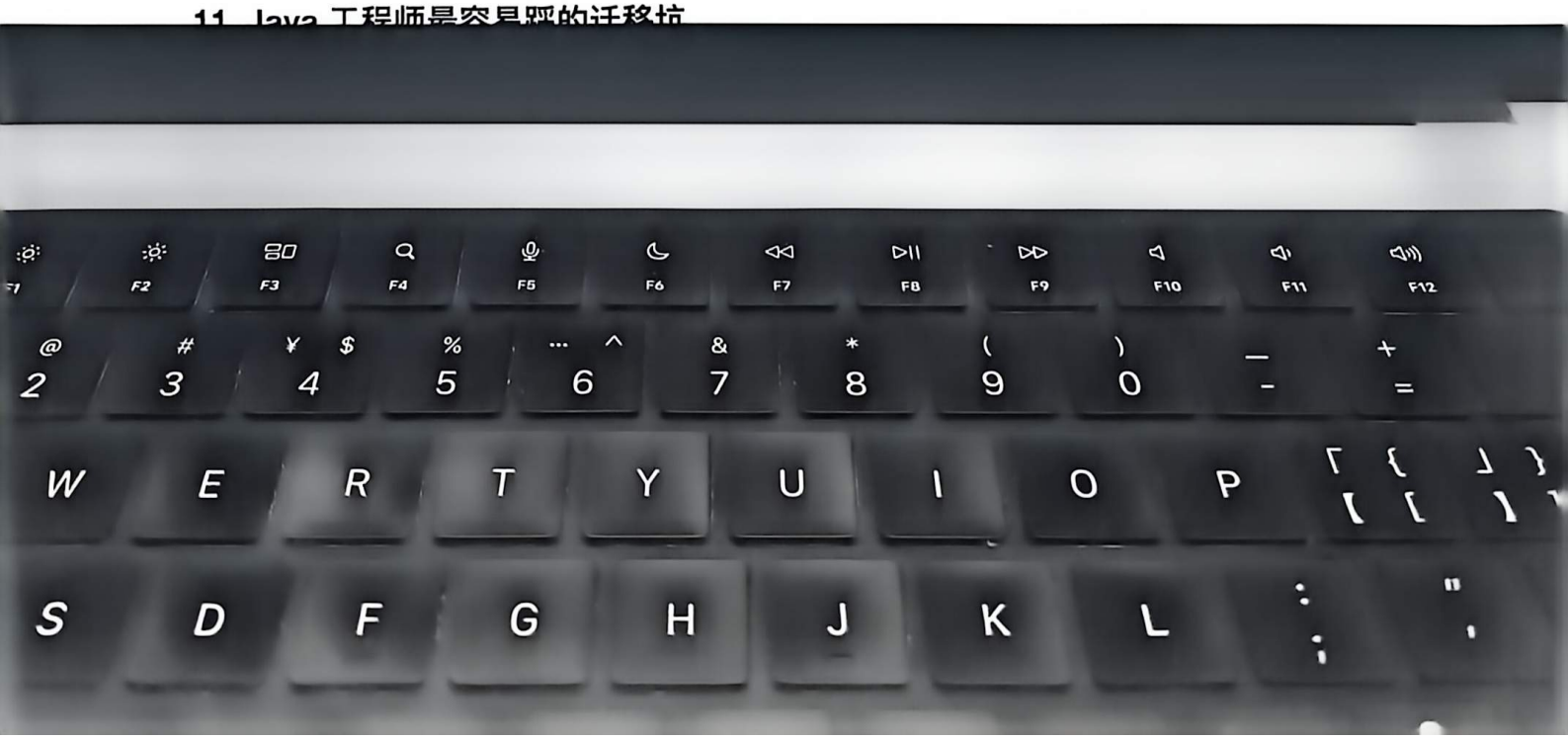
调用 `token_stream()` 得到 async generator 对象，它不可直接 await；消费者要使用 `async for`、`anext()`，并在提前停止时处理 `aclose()`。

如果把它交给普通 coroutine wrapper，wrapper 中的 `await function(...)` 会运行时报错。真正的流式观测层必须先定义：

- 耗时结束于第一项、自然耗尽还是 `aclose()` 完成；
- 消费者 `break`、异常、取消时谁关闭上游；
- 是否记录每一项，如何避免高基数和 token 内容泄露；
- 是否完整代理 `asend()`、`athrow()` 等协议。

这不是加一个 `inspect.isasyncgenfunction()` 分支就能自动正确。同步函数、一次返回的 coroutine 和逐项返回的 async generator，应该分别设计范围明确的 wrapper。

11. Java 工程师最容易踩的迁移坑



README README
aders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-2076f10863aad887aeeac8dc56e8.html
2.50
接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

11. Java 工程师最容易踩的迁移坑

写法或假设	失败表现	正确方向
模块全局保存当前 request ID	并发 Task 的日志串请求	请求入口设置 ContextVar
把 ContextVar 当作普通 ThreadLocal	错判 Task、线程、进程和网络边界	针对具体 API 核对传播合同
`finally: REQUEST_ID.set("-")`	嵌套调用退出后丢失真实外层值	保存 Token 并 `reset(token)`
先 create_task，后设置 request ID	子 Task 已经复制旧 Context	先 set，再创建子 Task
ContextVar 中放共享 dict 并原地修改	父子 Task 仍观察到彼此修改	优先放不可变值，或复制后重新 set
假设所有 executor 自动传播 Context	工作线程读取默认值或旧值	用 `to_thread` 的合同，或显式 `copy_context().run`
把 client/session 藏进 ContextVar	生命周期、事务和测试依赖不可见	明确传参或交给有所有权的组件
同步 wrapper 计时 async 函数	只测到 coroutine 创建	`async def wrapper` 并 await 原调用
success 写在无条件 `finally`	失败和取消也被记成成功	success 写在 `else`，finally 只做清理
`except BaseException: return None`	取消被吞，结构化并发状态失真	普通异常与取消分别记录并继续传播
日志打印全部 args/请求体/token	敏感数据泄露、字段基数失控	只记录审查过的 ID、状态和有界计数
注册器先于 wrapper 保存原函数	动态调用绕过日志和计时	先包装，再让外层注册 wrapper
用普通 async wrapper 包 async generator	generator 不可 await，关闭语义缺失	为流协议单独设计观测层

README × README ×
ders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-2076f10863aad887aeeac8dc56e8.html
的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

12. 运行本章完整实验

环境和依赖

仓库根目录的 `pyproject.toml` 要求 Python `>=3.14`，Ruff 与 Pyright 也以 Python 3.14 为目标。本章已核对的本地解释器是 **CPython 3.14.7**。

本章只使用标准库，不需要新增依赖、API Key、外网服务或日志平台。Python 3.14 特有的正文写法主要有 `with CONTEXT_VAR.set(...)`；核心生命周期仍可用显式 `set/reset` 理解。

本章目录包含：

文件	职责
`README.md`	Context 传播模型、Java 对照、日志与 wrapper 设计
`example.py`	10 组使用 Event 控制顺序的可运行实验

从仓库根目录执行

```
.venv/bin/python -B java_to_python/chapters/26_contextvars_observability/example.py
```

没有虚拟环境但已安装 Python 3.14 时，也可以执行：

```
python3.14 -B java_to_python/chapters/26_contextvars_observability/example.py
```

现有 `java_to_python/run_all.py` 只负责前 10 章，本章没有接入它。

当前环境已经提供开发工具时，可在仓库根目录继续运行：

```
.venv/bin/ruff check java_to_python/chapters/26_contextvars_observability
.venv/bin/ruff format --check java_to_python/chapters/26_contextvars_observability
.venv/bin/pyright java_to_python/chapters/26_contextvars_observability
```

完整示例会依次验证：

1. 普通全局变量让 REQ-A 的两个工具稳定误读 REQ-B；
2. Token 在嵌套、异常安全的 `finally` 和 Python 3.14 `with` 写法中恢复正确外层；

README × README
ders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-2076f10863aad887aeeac8dc56e8.html
接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

完整示例会依次验证：

1. 普通全局变量让 REQ-A 的两个工具稳定误读 REQ-B；
2. Token 在嵌套、异常安全的 `finally` 和 Python 3.14 `with` 写法中恢复正确外层；
3. 子 Task 读取创建点 Context，父子后续 set/reset 互不回写；
4. Context 副本仍可能指向同一个可变 dict；
5. `to_thread` 的直接 await、Task 调度和 `run_in_executor` 入口表现不同；
6. 同步 wrapper 返回时原协程尚未启动，异步 wrapper 保留签名并覆盖完整 await；
7. 两个请求各派生两个 tool Task，日志逐条匹配所属请求；
8. 普通失败记录 error，不记录 success，并传播原异常；
9. 取消在工具层和请求层记录 cancelled，不记录 success，并到达最外层；
10. async generator 不可直接 await，提前停止时需要显式关闭。

实验使用 `asyncio.Event` 建立明确先后关系，不靠固定 `sleep` 猜调度顺序。耗时数值、线程名和不同请求日志的打印顺序不是稳定合同。

本地替身只证明当前 Python 运行时中的 Task Context、线程适配、异常与取消语义。它没有证明真实 HTTP SDK、数据库驱动、跨进程队列、远端 trace header 或日志平台会自动传播同一上下文。

13. 递进练习

练习 1：增加租户诊断字段

新增模块级 `TENANT_LABEL: ContextVar[str]`，让日志 Filter 输出 `tenant=<value>`。两个并发请求分别使用 `team-blue` 和 `team-green`。

验收标准：

- 每条工具日志的 tenant 与所属请求一致；
- 请求结束后恢复默认值；
- 不修改工具签名来重复传 tenant；
- 授权逻辑仍不依赖这个诊断标签。

练习 2：修复可变上下文共享

保留共享 dict 反例，再新增一个正例。父 Context 初始值为 `{"events": ["parent"]}`，两个子 Task 分别构造新的 dict 和 list，再追加 `child-A`、`child-B` 并临时 set。

README README
/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-2076f10863aad887aeeac8dc56e8.html
需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

验收标准：

- 父对象最后没有变化；
- A 只看到 `parent, child-A`，B 只看到 `parent, child-B`；
- 用 `is` 或 `id()` 证明三个 dict、三个 list 不是同一对象；
- 每个子 Task 都在 `finally` 中 reset，顺序验证不依赖 `sleep`。

练习 3：封装显式线程传播

实现 `submit_with_current_context(executor, function, *args)`：每次调用都执行一次 `copy_context()`，再通过 `run_in_executor` 提交。用两个请求 Task 各提交两个同步读取函数。

验收标准：

- 四个线程结果都带正确请求 ID；
- 同一个 executor 可以复用，并由明确所有者关闭；
- 参数与返回类型关系使用参数规格和类型变量表达；
- 不缓存同一个 Context 给多个并发线程共用。

练习 4：给异步观测器增加 operation

让 `observe_async` 接收 `operation` 参数，并分别记录 started、success、error、cancelled。准备一个成功工具、一个抛 `ValueError` 的工具和一个停在 Event 上供外部取消的工具。

验收标准：

- 每次调用只有一个正确终态；
- 失败方仍收到同一个 `ValueError`，取消方仍收到 `CancelledError`；
- 终态日志有非负 `elapsed_ms`，但测试不限制精确毫秒数；
- `inspect.signature()` 与原签名一致，敏感参数没有进入日志。

练习 5：先定义 async generator 的观测合同

先写清自然耗尽、消费者提前关闭、上游异常和外部取消四条路径的结束点，再为一个两项异步生成器实现专用 wrapper。

验收标准：

- 四条路径都能有限结束，上游 `finally` 恰好执行一次；
- 异常和取消保持原协议，不转成普通结果；

README × README
7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-2076f10863aad887aeeac8dc56e8.html
需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

验收标准：

- 每次调用只有一个正确终态；
- 失败方仍收到同一个 `ValueError`，取消方仍收到 `CancelledError`；
- 终态日志有非负 `elapsed_ms`，但测试不限制精确毫秒数；
- `inspect.signature()` 与原签名一致，敏感参数没有进入日志。

练习 5：先定义 async generator 的观测合同

先写清自然耗尽、消费者提前关闭、上游异常和外部取消四条路径的结束点，再为一个两项异步生成器实现专用 wrapper。

验收标准：

- 四条路径都能有限结束，上游 `finally` 恰好执行一次；
- 异常和取消保持原协议，不转成普通结果；
- 不记录每个 token 的原文；
- 明确是否支持 `asend()` / `athrow()`，不要把有限实现描述成万能 wrapper。

14. 本章小结

可以把整章压缩成六条规则：

1. 普通全局变量按模块共享，单线程 asyncio 也会因为 `await` 交错而串请求。
2. ContextVar 按当前 Context 绑定；请求入口先 set，所有出口都用原 Token reset。
3. 子 Task 在创建点复制 Context，后续绑定互不回写，但可变 value 仍可能共享身份。
4. Task、线程、进程和网络是不同传播边界；只相信具体 API 的明确合同。
5. 日志关联字段适合隐式上下文，业务输入、资源和安全决策仍应显式建模。
6. 异步 wrapper 必须 await 原调用，并区分 success、error、cancelled；流式协议另行设计。

最后尝试不看正文回答：

1. 一条 tool 日志里的 request ID 在哪里设置，何时复制，最后由谁读取？
2. 为什么 `reset(token)` 比 `set(default)` 更能保护嵌套调用？
3. 为什么父子 Task 的 request ID 不串，ContextVar 里的 dict 却仍可能相互影响？
4. `to_thread(...)` 只创建 coroutine 后再改 Context，与先创建 Task 后再改，有什么不同？
5. 为什么同步 wrapper 的耗时不包含 I/O，成功日志也不能写在无条件 `finally` 中？
6. 为什么 async generator 不能直接复用“一次 await 得到 R”的 wrapper？

```python
"""第 26 章：contextvars、日志与异步装饰器。

所有并发顺序都由 Event 或已经发生的状态控制；耗时只用于观察，不做精确数值断言。
示例只使用标准库，不连接真实 API、数据库、日志平台或分布式追踪服务。
"""

# 推荐按编号从 1 读到 10：前五节先建立 Context 的传播模型，后五节再把它用于日志与装饰器。
# 整个文件的数据流是“请求 Task 设置上下文 -> 子 Task/线程读取 -> Filter 注入日志 -> 断言核对”。
# Event 负责规定并发步骤的先后，因此实验结果来自明确的 happens-before 关系，而不是碰巧的调度。

# asyncio 提供 Task、Event、取消和线程适配；contextvars 提供当前 Context 的复制操作。
# inspect 用来验证装饰器元信息及 async generator 协议，logging 则承载可检查的观测记录。
import asyncio
import contextvars
import inspect
import logging
import sys
import threading

# collections.abc 中的类型描述“可调用对象最终返回什么协议”；后文会特别区分 Awaitable、
# Coroutine 和 AsyncGenerator。其余导入分别用于线程池、上下文变量、数据载体、装饰器和计时。
from collections.abc import AsyncGenerator, Awaitable, Callable, Coroutine
from concurrent.futures import ThreadPoolExecutor
from contextvars import ContextVar, Token
from dataclasses import dataclass
from functools import wraps
from time import perf_counter_ns
from typing import Any, cast

# ContextVar 应在模块顶层声明：Context 会持有变量本身的引用，不应在每次请求的闭包里重复创建。
# `ContextVar[str]` 是给类型检查器看的值类型；第一个字符串是诊断名称，default 是未绑定时的值。
# 这三个变量对象是全局共享的，但“变量在当前 Context 中绑定了什么值”会随 Task/Context 改变。
REQUEST_ID: ContextVar[str] = ContextVar[str]("request_id", default="-")
RUN_ID: ContextVar[str] = ContextVar[str]("run_id", default="-")
TOOL_NAME: ContextVar[str] = ContextVar[str]("tool_name", default="-")

# 这个变量故意没有默认值：使用前必须先 set，否则 get() 会抛 LookupError。
```

```python
# 这个变量故意没有默认值：使用前必须先 set，否则 get() 会抛 LookupError。
# 它保存可变 dict，用于第 4 节证明复制 Context 不等于深拷贝其中的值。
MUTABLE_BAG: ContextVar[dict[str, list[str]]] = ContextVar[dict[str, list[str]]]("mutable_bag")

# 教学反例：模块全局名字只有一个进程级绑定，所有 asyncio Task 都读写同一个值。
# 单线程只保证同一瞬间执行一段 Python 代码，不会在 await 切换 Task 时保存并恢复普通全局变量。
GLOBAL_REQUEST_ID = "-"

# getLogger 取得命名 logger；模块级列表则由自定义 Handler 收集同一批 LogRecord，供后文断言。
# 列表也是共享可变对象，但这里所有追加和读取都在事件循环线程内，并且只服务于受控实验。
LOGGER: Logger = logging.getLogger("chapter26")
CAPTURED_RECORDS: list[logging.LogRecord] = []


# 集中打印章节标题。f-string 中的 `{title}` 会在调用时求值并嵌入字符串。
def print_section(title: str) -> None:
    print(f"\n=== {title} ===")


class ContextFieldsFilter(logging.Filter):
    """在输出边界把当前上下文字段补到每条 LogRecord。"""

    def filter(self, record: logging.LogRecord) -> bool:
        # logging 把一条日志的标准字段和 extra 字段都保存在 LogRecord 上。
        # 先用 hasattr 判断，是为了让调用方通过 extra 显式给出的字段优先；普通业务日志才从
        # 当前 Context 补值。`record.__dict__[name] = value` 会让 Formatter 的 `%(name)s` 可用。
        if not hasattr(record, "request_id"):
            record.__dict__["request_id"] = REQUEST_ID.get()
        if not hasattr(record, "run_id"):
            record.__dict__["run_id"] = RUN_ID.get()
        if not hasattr(record, "tool_name"):
            record.__dict__["tool_name"] = TOOL_NAME.get()

        if not hasattr(record, "task_name"):
            # current_task() 读取“当前运行的 asyncio Task”。它要求当前线程中有运行中的事件循环；
            # 在线程池工作线程里调用会抛 RuntimeError，而不是返回一个虚构的 Task。
```

```
56  class ContextFieldsFilter(logging.Filter):
59      def filter(self, record: logging.LogRecord) -> bool:
69
70          if not hasattr(record, "task_name"):
71              # current_task() 读取“当前运行的 asyncio Task”。它要求当前线程中有运行中的事件循环；
72              # 在线程池工作线程里调用会抛 RuntimeError，而不是返回一个虚构的 Task。
73              try:
74                  task: Task[Any] | None = asyncio.current_task()
75              except RuntimeError:
76                  # 工作线程没有正在运行的事件循环，所以不存在 asyncio Task。
77                  task = None
78
79              # 条件表达式读作“有 Task 就取 Task 名，否则记录当前线程名”。这相当于 Java 的
80              # `condition ? left : right`，但 Python 把条件写在两个结果中间。
81              task_name: str = (
82                  task.get_name() if task is not None else f"thread:{threading.current_thread().name}"
83              )
84              record.__dict__["task_name"] = task_name
85
86          # logging.Filter 返回 True 表示保留这条记录；返回 False 才会把它丢弃。
87          return True
88
89
90  class CapturingHandler(logging.Handler):
91      """保存记录，供示例用状态断言验证日志归属和取消状态。"""
92
93      def emit(self, record: logging.LogRecord) -> None:
94          # Handler 接收到的是 LogRecord 对象，不是已经格式化好的文本；后面可以检查字段和消息。
95          CAPTURED_RECORDS.append(record)
96
97
98  def configure_logging() -> None:
99      """只配置本章自己的 logger，不改应用的 root logger。"""
100
101     # 先清理这个命名 logger 和捕获列表，使重复调用 main() 时不会叠加 Handler 或旧记录。
102     LOGGER.handlers.clear()
103     LOGGER.filters.clear()
```

```python
def configure_logging() -> None:
    """只配置本章自己的 logger，不改应用的 root logger。"""

    # 先清理这个命名 logger 和捕获列表，使重复调用 main() 时不会叠加 Handler 或旧记录。
    LOGGER.handlers.clear()
    LOGGER.filters.clear()
    CAPTURED_RECORDS.clear()

    # INFO 让本章的 info/warning/error 都可见；propagate=False 阻止记录再次交给 root logger，
    # 否则使用者已经配置 root Handler 时，同一条日志可能输出两次。
    LOGGER.setLevel(logging.INFO)
    LOGGER.propagate = False
    LOGGER.addFilter(ContextFieldsFilter())

    # StreamHandler 把日志写到标准输出。Formatter 中的 `%(request_id)s` 等占位符依赖上面的
    # Filter 先补齐字段；`%(message)s` 则是 LOGGER.info(...) 的模板完成参数替换后的正文。
    console: StreamHandler[TextIO | Any] = logging.StreamHandler(sys.stdout)
    console.setFormatter(
        logging.Formatter(
            "%(levelname)s request=%(request_id)s run=%(run_id)s "
            "task=%(task_name)s tool=%(tool_name)s | %(message)s"
        )
    )

    # 同一 LogRecord 先输出到终端，再交给 CapturingHandler 保存，形成“给人看 + 给断言查”两路。
    LOGGER.addHandler(console)
    LOGGER.addHandler(CapturingHandler())


@dataclass(slots=True)
class ToolGate:
    """用显式事件控制工具何时开始等待、何时继续。"""

    # dataclass 会根据这两个注解生成初始化方法；字段保存 Event 对象本身。
    # slots=True 不创建普通实例 __dict__，但这只是紧凑的数据载体，不是 Java 字段的静态类型约束。
    started: asyncio.Event
```

```python
@dataclass(slots=True)
class ToolGate:
    """用显式事件控制工具何时开始等待、何时继续。"""

    # dataclass 会根据这两个注解生成初始化方法；字段保存 Event 对象本身。
    # slots=True 不创建普通实例 __dict__，但这只是紧凑的数据载体，不是 Java 字段的静态类型约束。
    started: asyncio.Event
    may_finish: asyncio.Event

    @classmethod
    def create(cls) -> ToolGate:
        # 类方法收到类对象 cls，因此子类调用时也能构造子类；每次调用都创建两个全新的 Event。
        return cls(asyncio.Event(), asyncio.Event())


@dataclass(frozen=True, slots=True)
class ToolObservation:
    # 这个不可变结果对象把“工具标签、实际看到的请求和运行标识”一起交回调用方。
    # frozen=True 阻止普通字段重新赋值，类似一个轻量只读 DTO，但 Python 类型注解仍主要供静态检查。
    label: str
    observed_request_id: str
    observed_run_id: str


# ---------------------------------------------------------------------------
# 1. 全局变量为何在单线程 asyncio 中串值
# ---------------------------------------------------------------------------
# 工具先报告“我已启动”，再等待测试代码放行；放行后才读取全局变量。
# 这确保读取发生在 REQ-B 覆盖全局值之后，使反例稳定可重复。
async def globally_scoped_tool(label: str, gate: ToolGate) -> ToolObservation:
    gate.started.set()
    await gate.may_finish.wait()
    observed: str = GLOBAL_REQUEST_ID

    # `%s` 是 logging 的延迟参数格式化；extra 明确指定本条反例日志的三个上下文字段，
    # 因此 Filter 不会拿 ContextVar 默认值覆盖它们。
```

```python
# ---------------------------------------------------------------------------
# 1. 全局变量为何在单线程 asyncio 中串值
# ---------------------------------------------------------------------------
# 工具先报告"我已启动"，再等待测试代码放行；放行后才读取全局变量。
# 这确保读取发生在 REQ-B 覆盖全局值之后，使反例稳定可重复。
async def globally_scoped_tool(label: str, gate: ToolGate) -> ToolObservation:
    gate.started.set()
    await gate.may_finish.wait()
    observed: str = GLOBAL_REQUEST_ID

    # `%s` 是 logging 的延迟参数格式化；extra 明确指定本条反例日志的三个上下文字段，
    # 因此 Filter 不会拿 ContextVar 默认值覆盖它们。
    LOGGER.info(
        "全局反例读取上下文 label=%s",
        label,
        extra={
            "request_id": observed,
            "run_id": "bad-global",
            "tool_name": "globally_scoped_tool",
        },
    )
    # 返回读取瞬间的快照；后续全局变量即使再变，也不会改写这个 str 字段。
    return ToolObservation(label, observed, "bad-global")


async def run_bad_request(
    request_id: str,
    *,
    first_request_has_set: asyncio.Event,
    second_request_has_set: asyncio.Event,
    is_first: bool,
) -> tuple[ToolObservation, ...]:
    # `global` 声明下面的赋值要改模块级名字；没有它，赋值会创建同名局部变量。
    # 它不会提供锁、Task 隔离或 Java volatile 的可见性语义。
    global GLOBAL_REQUEST_ID

    # 两个请求运行同一函数，由 is_first 选择不同的握手顺序：A 先写，B 等到 A 写完后覆盖。
```

```python

async def run_bad_request(
    request_id: str,
    *,
    first_request_has_set: asyncio.Event,
    second_request_has_set: asyncio.Event,
    is_first: bool,
) -> tuple[ToolObservation, ...]:
    # `global` 声明下面的赋值要改模块级名字；没有它，赋值会创建同名局部变量。
    # 它不会提供锁、Task 隔离或 Java volatile 的可见性语义。
    global GLOBAL_REQUEST_ID

    # 两个请求运行同一函数，由 is_first 选择不同的握手顺序：A 先写，B 等到 A 写完后覆盖。
    if is_first:
        GLOBAL_REQUEST_ID: str = request_id
        first_request_has_set.set()
        # 明确等到另一个请求覆盖全局值，再创建本请求的工具 Task。
        await second_request_has_set.wait()
    else:
        await first_request_has_set.wait()
        GLOBAL_REQUEST_ID: str = request_id
        second_request_has_set.set()

    # tuple 中每个工具各有一对 Event，避免一个工具的状态替另一个工具“提前报到”。
    gates: tuple[ToolGate, ToolGate] = (ToolGate.create(), ToolGate.create())

    # 这是生成器表达式交给 tuple(...) 收集：enumerate(..., start=1) 每轮给出序号和 gate，
    # create_task 立即把 coroutine 包成可调度 Task，并设置便于日志核对的名字。
    tasks: tuple[Task[ToolObservation], ...] = tuple(
        asyncio.create_task(
            globally_scoped_tool(f"{request_id}-tool-{index}", gate),
            name=f"global:{request_id}:{index}",
        )
        for index, gate in enumerate(gates, start=1)
    )

    # `...` 在调用位置把等待对象逐项展开给 gather：先等所有工具到达屏障，再逐个 set 放行。
```

```
176  async def run_bad_request(
210
211      # `*` 在调用位置把等待对象逐项展开给 gather；先等所有工具到达屏障，再逐个 set 放行。
212      await asyncio.gather(*(gate.started.wait() for gate in gates))
213      for gate in gates:
214          gate.may_finish.set()
215
216      # 第二次 gather 等两个工具完成并按传入顺序返回结果；外层 tuple 固定返回容器类型。
217      return tuple(await asyncio.gather(*tasks))
218
219
220  async def demonstrate_global_leak() -> None:
221      print_section("1. 反例：单线程中的全局 request_id 仍会串值")
222      global GLOBAL_REQUEST_ID
223
224      # 保存实验前的真实模块状态，并为两个请求的覆盖动作创建握手 Event。
225      previous_value: str = GLOBAL_REQUEST_ID
226      first_has_set = asyncio.Event()
227      second_has_set = asyncio.Event()
228      try:
229          # 两个请求是并发 Task；关键字参数把两个 Event 明确绑定到对应形参，便于读出协议。
230          request_a: Task[tuple[ToolObservation, ...]] = asyncio.create_task(
231              run_bad_request(
232                  "REQ-A",
233                  first_request_has_set=first_has_set,
234                  second_request_has_set=second_has_set,
235                  is_first=True,
236              ),
237              name="global-request:A",
238          )
239          request_b: Task[tuple[ToolObservation, ...]] = asyncio.create_task(
240              run_bad_request(
241                  "REQ-B",
242                  first_request_has_set=first_has_set,
243                  second_request_has_set=second_has_set,
244                  is_first=False,
```

```python
async def demonstrate_global_leak() -> None:
        )
        request_b: Task[tuple[ToolObservation, ...]] = asyncio.create_task(
            run_bad_request(
                "REQ-B",
                first_request_has_set=first_has_set,
                second_request_has_set=second_has_set,
                is_first=False,
            ),
            name="global-request:B",
        )

        # gather 的两个返回值按 request_a、request_b 的输入顺序解包，与实际完成先后无关。
        results_a: tuple[ToolObservation, ...], results_b: tuple[ToolOb… = await asyncio.gather(request_a, requ
    finally:
        # finally 在正常、异常和取消时都会执行，让反例不污染后面的正确示例。
        GLOBAL_REQUEST_ID: str = previous_value

    # 列表推导式从每个不可变观察结果提取 request_id，等价于循环 append，但更直接描述结果。
    observed_a: list[str] = [item.observed_request_id for item in results_a]
    observed_b: list[str] = [item.observed_request_id for item in results_b]
    print("  REQ-A 的两个工具实际读到:", observed_a)
    print("  REQ-B 的两个工具实际读到:", observed_b)

    # A、B 都读到 B，证明 await 处切换 Task 后，普通模块全局值不会随请求自动恢复。
    # 最后一条同时验证本节已经恢复进入实验前的状态。
    assert observed_a == ["REQ-B", "REQ-B"]
    assert observed_b == ["REQ-B", "REQ-B"]
    assert GLOBAL_REQUEST_ID == previous_value


# ---------------------------------------------------------------------------
# 2. get / set / reset / token 与嵌套恢复
# ---------------------------------------------------------------------------
# 这一节只在当前同步调用栈中嵌套改值，专门观察 Token 如何精确恢复上一层绑定。
def demonstrate_token_lifecycle() -> None:
    print_section("2. ContextVar: set 返回 token，finally 按层恢复")
```

```
268  # ------------------------------------------------------------------------
269  # 2. get / set / reset / token 与嵌套恢复
270  # ------------------------------------------------------------------------
271  # 这一节只在当前同步调用栈中嵌套改值，专门观察 Token 如何精确恢复上一层绑定。
272  def demonstrate_token_lifecycle() -> None:
273      print_section("2. ContextVar: set 返回 token, finally 按层恢复")
274
275      # get() 读取当前 Context 的绑定；入口尚未显式绑定，所以这里得到声明时的默认值 "-"。
276      initial: str = REQUEST_ID.get()
277
278      # set() 不是修改 ContextVar 对象里的一个全局字段，而是在当前 Context 建立新绑定，
279      # 并返回记录旧状态的 Token。Token 是后续 reset 的恢复凭证。
280      outer_token: Token[str] = REQUEST_ID.set("REQ-OUTER")
281      try:
282          print("  外层值:", REQUEST_ID.get())
283
284          # `is` 比较对象身份。Token.MISSING 是专用哨兵，表示 set 前根本没有显式绑定；
285          # 这与“旧值恰好是默认字符串 '-'”不是同一状态。
286          print("  外层此前没有显式绑定:", outer_token.old_value is Token.MISSING)
287
288          # 第二次 set 暂时遮住外层值。嵌套层必须各自保存自己的 Token。
289          inner_token: Token[str] = REQUEST_ID.set("REQ-INNER")
290          try:
291              print("  嵌套覆盖后:", REQUEST_ID.get())
292          finally:
293              # reset(inner_token) 不是写死默认值，而是恢复这次 set 前的准确外层值 REQ-OUTER。
294              # 放在 finally 中，内层即使抛异常或收到取消也不会泄漏 REQ-INNER。
295              REQUEST_ID.reset(inner_token)
296          print("  内层退出，恢复外层:", REQUEST_ID.get())
297
298          # Python 3.14 起 Token 也实现同步上下文管理器协议。这里的表达式仍先执行 set，
299          # with 退出时再用所得 Token 自动 reset；它是同一生命周期的缩写，不是新隔离机制。
300          with REQUEST_ID.set("REQ-3.14-WITH"):
301              print("  3.14 with token 内:", REQUEST_ID.get())
302          print("  3.14 with token 后:", REQUEST_ID.get())
303      finally:
```

```
272  def demonstrate_token_lifecycle() -> None:
303      finally:
304          # 后 set 的内层已经先恢复，现在再消费 outer_token，按栈的反方向退回初始状态。
305          REQUEST_ID.reset(outer_token)
306
307      print("  整段结束，恢复初始值:", REQUEST_ID.get())
308
309      # Python 支持链式比较：a == b == c 等价于同时验证 a == b 且 b == c。
310      assert REQUEST_ID.get() == initial == "-"
311
312
313  # ---------------------------------------------------------------------------
314  # 3. Task 创建时复制 Context；后续 set 只改各自 Context
315  # ---------------------------------------------------------------------------
316  @dataclass(frozen=True, slots=True)
317  class TaskContextObservation:
318      # 三个字段分别记录子 Task 初次读取、局部覆盖、reset 后的值，使时序结果可一次性比较。
319      at_child_start: str
320      after_child_set: str
321      after_child_reset: str
322
323
324  async def child_context_probe(
325      started: asyncio.Event,
326      may_change: asyncio.Event,
327  ) -> TaskContextObservation:
328      # 子 Task 第一次运行时，从它在 create_task 时取得的 Context 副本中读值。
329      inherited: str = REQUEST_ID.get()
330
331      # 通知父 Task “子任务已读取初始值”，然后等待父 Task 明确放行局部修改。
332      started.set()
333      await may_change.wait()
334
335      # 子 Task 的 set 只改自己的当前 Context，不会回写父 Task 的 Context。
336      child_token: Token[str] = REQUEST_ID.set("REQ-CHILD-LOCAL")
337      try:
```

```
async def child_context_probe(
    # 子 Task 的 set 只改自己的当前 context，不会回写父 Task 的 context。
    child_token: Token[str] = REQUEST_ID.set("REQ-CHILD-LOCAL")
    try:
        child_local: str = REQUEST_ID.get()
    finally:
        REQUEST_ID.reset(child_token)

    # 最后一个 get 发生在 reset 之后，应重新看见创建 Task 时继承的值。
    return TaskContextObservation(inherited, child_local, REQUEST_ID.get())


async def demonstrate_task_context_copy() -> None:
    print_section("3. Task 在创建时复制上下文，父子后续修改相互隔离")

    # 外层绑定用于证明多层 reset 最终能完整还原；内层 before_create 则是子 Task 要继承的值。
    initial_token: Token[str] = REQUEST_ID.set("REQ-PARENT-OUTER")
    try:
        before_create_token: Token[str] = REQUEST_ID.set("REQ-BEFORE-CREATE")
        try:
            started = asyncio.Event()
            may_change = asyncio.Event()

            # create_task 在此刻复制当前 Context。即使事件循环还没执行 child_context_probe，
            # child 也已经持有 REQUEST_ID=REQ-BEFORE-CREATE 的独立 Context 映射。
            child: Task[TaskContextObservation] = asyncio.create_task(
                child_context_probe(started, may_change),
                name="context-copy-child",
            )

            # 父 Task 随后改成 REQ-AFTER-CREATE，只影响父 Task 当前 Context，不会追溯更新 child。
            after_create_token: Token[str] = REQUEST_ID.set("REQ-AFTER-CREATE")
            try:
                # 等子 Task 确实读过继承值后再放行；await child 取得的就是它 return 的数据对象。
                await started.wait()
                may_change.set()
                child_values: TaskContextObservation = await child
```

```
346  async def demonstrate_task_context_copy() -> None:
366              try:
367                  # 等子 Task 确实读过继承值后再放行；await child 取得的就是它 return 的数据对象。
368                  await started.wait()
369                  may_change.set()
370                  child_values: TaskContextObservation = await child
371                  parent_value: str = REQUEST_ID.get()
372              finally:
373                  # 三层绑定严格逆序恢复：after_create -> before_create -> initial。
374                  REQUEST_ID.reset(after_create_token)
375          finally:
376              REQUEST_ID.reset(before_create_token)
377      finally:
378          REQUEST_ID.reset(initial_token)
379
380      print("  子 Task 开始时:", child_values.at_child_start)
381      print("  子 Task 自己 set 后:", child_values.after_child_set)
382      print("  子 Task reset 后:", child_values.after_child_reset)
383      print("  父 Task 在子 Task 修改后:", parent_value)
384
385      # dataclass 自动生成按字段比较的 __eq__，所以可直接把整个观察结果与期望对象比较。
386      assert child_values == TaskContextObservation(
387          "REQ-BEFORE-CREATE",
388          "REQ-CHILD-LOCAL",
389          "REQ-BEFORE-CREATE",
390      )
391      assert parent_value == "REQ-AFTER-CREATE"
392      assert REQUEST_ID.get() == "-"
393
394
395  # ------------------------------------------------------------------------
396  # 4. 复制 Context 不会深拷贝 ContextVar 中的值
397  # ------------------------------------------------------------------------
398  # 第一个子 Task 取得共享 dict 后原地 append，再通知第二个子 Task；返回 id 用于比较对象身份。
399  async def append_first(shared_changed: asyncio.Event, peer_changed: asyncio.Event) -> int:
400      bag: dict[str, list[str]] = MUTABLE_BAG.get()
401      bag["events"].append("child-A")
```

```python
# ------------------------------------------------------------------------------
# 4. 复制 Context 不会深拷贝 ContextVar 中的值
# ------------------------------------------------------------------------------
# 第一个子 Task 取得共享 dict 后原地 append，再通知第二个子 Task；返回 id 用于比较对象身份。
async def append_first(shared_changed: asyncio.Event, peer_changed: asyncio.Event) -> int:
    bag: dict[str, list[str]] = MUTABLE_BAG.get()
    bag["events"].append("child-A")
    shared_changed.set()
    await peer_changed.wait()
    return id(bag)


# 第二个子 Task 必须等 A 修改完成才 append，因此最终列表顺序是确定的，不依赖调度运气。
async def append_second(shared_changed: asyncio.Event, peer_changed: asyncio.Event) -> int:
    await shared_changed.wait()
    bag: dict[str, list[str]] = MUTABLE_BAG.get()
    bag["events"].append("child-B")
    peer_changed.set()
    return id(bag)


async def demonstrate_shallow_context_copy() -> None:
    print_section("4. Context 的映射被复制，可变值本身没有深拷贝")

    # 字典字面量创建一个 dict，其中 events 指向可变 list；类型可由 MUTABLE_BAG 的注解推断。
    shared_bag: dict[str, list[Any]] = {"events": []}

    # 父 Task 先绑定这个对象，随后创建的两个子 Task 都会复制“变量 -> 该对象”的映射。
    bag_token: Token[dict[str, list[str]]] = MUTABLE_BAG.set(shared_bag)
    try:
        # copy_context() 创建另一份 Context 映射快照，但不会 deepcopy 映射中的 dict/list。
        snapshot: Context = contextvars.copy_context()
        first_changed = asyncio.Event()
        second_changed = asyncio.Event()
        task_a: Task[int] = asyncio.create_task(
            append_first(first_changed, second_changed),
```

```python
async def demonstrate_shallow_context_copy() -> None:
        task_a: Task[int] = asyncio.create_task(
            append_first(first_changed, second_changed),
            name="mutable-context:A",
        )
        task_b: Task[int] = asyncio.create_task(
            append_second(first_changed, second_changed),
            name="mutable-context:B",
        )

        # gather 按 task_a、task_b 顺序交回两个 id；它不决定两个 Task 内部的运行顺序，
        # 内部顺序由 first_changed/second_changed 两个 Event 决定。
        first_identity: int, second_identity: int = await asyncio.gather(task_a, task_b)

        # Context 支持用 ContextVar 作为键读取快照值。`is` 比较身份，证明不是内容相同的副本。
        print("  copy_context 中仍是同一 dict:", snapshot[MUTABLE_BAG] is shared_bag)
        print("  父 Task 最终也看到:", shared_bag["events"])
        print("  父子引用身份相同:", id(shared_bag) == first_identity == second_identity)

        # 两个 append 都直接修改同一 list，所以父 Task 也看到 child-A、child-B。
        # ContextVar 隔离的是绑定，不附带深拷贝、锁或消息传递能力。
        assert snapshot[MUTABLE_BAG] is shared_bag
        assert shared_bag["events"] == ["child-A", "child-B"]
        assert id(shared_bag) == first_identity == second_identity
    finally:
        # 删除本节建立的显式绑定；因为变量没有 default，之后误用 get() 会继续及时暴露问题。
        MUTABLE_BAG.reset(bag_token)


# ---------------------------------------------------------------------------
# 5. asyncio.to_thread 的上下文复制时机和 API 边界
# ---------------------------------------------------------------------------
@dataclass(frozen=True, slots=True)
class ThreadObservation:
    # 工作线程把标签、实际读到的 ContextVar 值和线程名作为普通结果传回事务循环。
    label: str
```

```python

# ---------------------------------------------------------------------------
# 5. asyncio.to_thread 的上下文复制时机和 API 边界
# ---------------------------------------------------------------------------
@dataclass(frozen=True, slots=True)
class ThreadObservation:
    # 工作线程把标签、实际读到的 ContextVar 值和线程名作为普通结果传回事务循环。
    label: str
    request_id: str
    thread_name: str


# 这是在线程池中运行的同步 callable；它不能 await，只在执行瞬间读取线程中的当前 Context。
def read_context_in_thread(label: str) -> ThreadObservation:
    return ThreadObservation(label, REQUEST_ID.get(), threading.current_thread().name)


async def demonstrate_to_thread_context() -> None:
    print_section("5. to_thread: 在 coroutine 真正开始时复制当前 Context")

    # 调用 to_thread(...) 只创建 coroutine 对象，尚未进入它的函数体，也尚未 copy_context。
    call_token: Token[str] = REQUEST_ID.set("REQ-WHEN-CALLED")
    try:
        deferred: CoroutineType[Any, Any, ThreadObservat..] = asyncio.to_thread(read_context_in_thread, "直接 await
        # 在真正 await 前改父 Task 的当前值；to_thread 开始执行时会复制这个较新的 Context。
        await_token: Token[str] = REQUEST_ID.set("REQ-WHEN-AWAITED")
        try:
            # 到这里直接 await，to_thread 的函数体才开始并把 Context 传播到工作线程。
            directly_awaited: ThreadObservation = await deferred
        finally:
            REQUEST_ID.reset(await_token)
    finally:
        REQUEST_ID.reset(call_token)

    task_token: Token[str] = REQUEST_ID.set("REQ-WHEN-TASK-CREATED")
    try:
```

```
473 async def demonstrate_to_thread_context() -> None:
492     try:
493         # create_task 先为承载 to_thread 的子 Task 复制当前 Context。子 Task 日后开始执行时，
494         # 再从自己的 Context 复制给工作线程，所以线程看到的是 Task 创建点的值。
495         thread_task: Task[ThreadObservation] = asyncio.create_task(
496             asyncio.to_thread(read_context_in_thread, "先创建 Task"),
497             name="to-thread-proxy",
498         )
499 
500         # 父 Task 此后改值不会回写已经创建的 thread_task；这里同时保存父 Task 自己看到的值。
501         parent_changed_token: Token[str] = REQUEST_ID.set("REQ-PARENT-CHANGED-LATER")
502         try:
503             scheduled_as_task: ThreadObservation = await thread_task
504             parent_after_creation: str = REQUEST_ID.get()
505         finally:
506             REQUEST_ID.reset(parent_changed_token)
507     finally:
508         REQUEST_ID.reset(task_token)
509 
510     executor_token: Token[str] = REQUEST_ID.set("REQ-EXECUTOR-CALLER")
511     try:
512         # get_running_loop 取得当前正在驱动本协程的事件循环；必须在 async 运行期间调用。
513         loop: AbstractEventLoop = asyncio.get_running_loop()
514 
515         # 新建专用池让实验不受别处工作线程残留状态影响；with 退出会等待并关闭线程池，
516         # 类似 Java try-with-resources 的所有权边界，但协议方法是 Python 的 __enter__/__exit__。
517         with ThreadPoolExecutor(max_workers=1, thread_name_prefix="chapter26") as executor:
518             # run_in_executor 直接提交同步函数，没有 to_thread 的自动 Context 传播合同，
519             # 所以新线程从默认值 "-" 开始，而不是继承 REQ-EXECUTOR-CALLER。
520             direct_executor: ThreadObservation = await loop.run_in_executor(
521                 executor,
522                 read_context_in_thread,
523                 "run_in_executor 直接提交",
524             )
525 
526             # 在事件循环线程复制当前 Context；下面的闭包持有这份快照，稍后在线程里调用 run。
```

```
473  async def demonstrate_to_thread_context() -> None:
525
526              # 在事件循环线程复制当前 Context；下面的闭包持有这份快照，稍后在线程里调用 run。
527              copied: Context = contextvars.copy_context()
528
529              def read_in_copied_context() -> ThreadObservation:
530                  # Context.run 临时让同步 callable 在 copied 中执行，从而显式传播 request_id。
531                  return copied.run(
532                      read_context_in_thread,
533                      "run_in_executor + Context.run",
534                  )
535
536              explicit_executor: ThreadObservation = await loop.run_in_executor(
537                  executor,
538                  read_in_copied_context,
539              )
540      finally:
541          REQUEST_ID.reset(executor_token)
542
543      print("  调用后改值、再直接 await，线程看到:", directly_awaited.request_id)
544      print("  create_task 后父 Task 改值，线程看到:", scheduled_as_task.request_id)
545      print("  同期父 Task 自己看到:", parent_after_creation)
546      print("  run_in_executor 直接提交，线程看到:", direct_executor.request_id)
547      print("  显式 Context.run 提交，线程看到:", explicit_executor.request_id)
548
549      # 五条断言分别固定两个复制时机、父 Task 的独立值和两种 executor 入口的传播差异。
550      # 这里只证明这些标准库 API 在当前版本的合同，不应推广为“所有线程库都会/都不会传播”。
551      assert directly_awaited.request_id == "REQ-WHEN-AWAITED"
552      assert scheduled_as_task.request_id == "REQ-WHEN-TASK-CREATED"
553      assert parent_after_creation == "REQ-PARENT-CHANGED-LATER"
554      assert direct_executor.request_id == "-"
555      assert explicit_executor.request_id == "REQ-EXECUTOR-CALLER"
556      assert REQUEST_ID.get() == "-"
557
558
559  # ------------------------------------------------------------------------
560  # 6、范围明确、类型关系保真的异步日志装饰器
```

```python

# ------------------------------------------------------------------------------
# 6. 范围明确、类型关系保真的异步日志装饰器
# ------------------------------------------------------------------------------
# `**P` 是参数规格：把原函数的整套位置/关键字参数关系传给 wrapper；R 是 await 后的结果类型。
# 输入只要求调用后得到 Awaitable[R]，输出写得更具体：async wrapper 调用后一定得到 Coroutine。
# 这两层是静态类型关系，不会在运行时替调用参数做 Java 编译器式校验。
def observe_async[**P, R](
    function: Callable[P, Awaitable[R]],
) -> Callable[P, Coroutine[Any, Any, R]]:
    """只包装“一次 await 得到一个最终结果”的异步可调用对象。"""

    # 装饰器应用时先执行 wraps(function)，把原函数的名字、docstring、注解和 __wrapped__ 链
    # 更新到 wrapper 上；它保留运行时元信息，参数规格 P 则保留静态调用关系，两者职责不同。
    @wraps(function)
    async def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        # 每次调用在自己的当前 Context 临时设置工具名；并发调用不会共享这个绑定。
        tool_token: Token[str] = TOOL_NAME.set(function.__name__)

        # perf_counter_ns 是适合计算时间间隔的单调高分辨率时钟；它不是业务时间戳。
        # 起点在调用原函数之前，因此后面的 elapsed 包含协程执行和 I/O 等待的墙钟时间。
        started_ns: int = perf_counter_ns()
        LOGGER.info("工具调用开始 status=started")
        try:
            # 计时范围包住 await，因此包含原协程函数体及其中真正的 I/O 等待。
            result: R = await function(*args, **kwargs)
        except asyncio.CancelledError:
            # CancelledError 表示协作式取消，不是普通业务失败。记录终态后裸 raise，
            # 让同一个取消继续到调用方；吞掉它会破坏 timeout、TaskGroup 等上层控制流。
            elapsed_ms: float = (perf_counter_ns() - started_ns) / 1_000_000
            LOGGER.warning("工具调用取消 status=cancelled elapsed_ms=%.3f", elapsed_ms)
            raise
        except Exception as error:
            # 这里只捕获普通 Exception。`as error` 绑定异常对象，用 type(...).__name__ 记录类别；
            # LOGGER.exception 会在当前 except 中附带 traceback，随后裸 raise 保留原异常合同。
            elapsed_ms: float = (perf_counter_ns() - started_ns) / 1_000_000
```

```
565  def observe_async[**P, R](
573      async def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
590          except Exception as error:
591              # 这里只捕获普通 Exception。`as error` 绑定异常对象，用 type(...).__name__ 记录类别；
592              # LOGGER.exception 会在当前 except 中附带 traceback，随后裸 raise 保留原异常合同。
593              elapsed_ms: float = (perf_counter_ns() - started_ns) / 1_000_000
594              # 受控示例可保留 traceback；生产环境仍需审查异常文本是否含敏感数据。
595              LOGGER.exception(
596                  "工具调用失败 status=error elapsed_ms=%.3f error_type=%s",
597                  elapsed_ms,
598                  type(error).__name__,
599              )
600              raise
601          else:
602              # try 的 else 只在 await 正常返回时执行，因此失败和取消不会被误记成 success。
603              elapsed_ms: float = (perf_counter_ns() - started_ns) / 1_000_000
604              LOGGER.info("工具调用成功 status=success elapsed_ms=%.3f", elapsed_ms)
605              return result
606          finally:
607              # 无论哪条终态路径都恢复进入 wrapper 前的工具名，支持装饰器嵌套，也避免泄漏到调用方。
608              TOOL_NAME.reset(tool_token)
609
610      # observe_async 在函数定义/装饰阶段返回 wrapper；真正的异步业务要到 wrapper 被调用并 await 才运行。
611      return wrapper
612
613
614  def wrong_sync_timing[**P, R](
615      function: Callable[P, Awaitable[R]],
616  ) -> Callable[P, Awaitable[R]]:
617      """教学反例：同步 wrapper 只测量了 coroutine 对象的创建。"""
618
619      # 这里故意使用普通 def。wraps 虽能保留元信息，却不能把同步控制流自动变成异步控制流。
620      @wraps(function)
621      def wrapper(*args: P.args, **kwargs: P.kwargs) -> Awaitable[R]:
622          started_ns: int = perf_counter_ns()
623
```

```
614  def wrong_sync_timing[**P, R](
619      # 这里故意使用普通 def。wraps 虽能保留元信息，却不能把同步控制流自动变成异步控制流。
620      @wraps(function)
621      def wrapper(*args: P.args, **kwargs: P.kwargs) -> Awaitable[R]:
622          started_ns: int = perf_counter_ns()
623
624          # 调用 async function 只创建并返回 coroutine/Awaitable；原函数体第一行尚未执行。
625          awaitable: Awaitable[R] = function(*args, **kwargs)
626
627          # 因为计时在任何 await 之前结束，它只覆盖对象创建开销，不包含实际工作或等待 I/O。
628          elapsed_ms: float = (perf_counter_ns() - started_ns) / 1_000_000
629          print(f"  [错误计时] coroutine 创建耗时={elapsed_ms:.3f} ms")
630          return awaitable
631
632      return wrapper
633
634
635  # 装饰器语法在定义阶段等价于 `gated_coroutine = wrong_sync_timing(gated_coroutine)`。
636  # 对外名字因此指向同步 wrapper，但调用后仍返回原 async 函数创建的 Awaitable。
637  @wrong_sync_timing
638  async def gated_coroutine(started: asyncio.Event, may_finish: asyncio.Event) -> str:
639      # 这两个 Event 让调用方观察原函数体何时真正开始，以及它是否仍停在等待点。
640      started.set()
641      await may_finish.wait()
642      return "真正的异步工作已完成"
643
644
645  # Python 3.12+ 的 type 语句定义类型别名。`...` 表示注册表容纳不同签名的函数；
646  # Awaitable[object] 诚实表达动态入口只能承诺“等待后得到某个对象”，不能恢复每项的精确 R。
647  type AnyAsyncTool = Callable[..., Awaitable[object]]
648
649
650  @dataclass(frozen=True, slots=True)
651  class RegisteredTool:
652      # 注册表的值同时保存外部名称与真正要调用的函数对象。
653      name: str
```

```python
@dataclass(frozen=True, slots=True)
class RegisteredTool:
    # 注册表的值同时保存外部名称与真正要调用的函数对象。
    name: str
    function: AnyAsyncTool


# 字符串键让调用方能动态选择工具，也意味着这里是静态类型信息被擦除的边界。
TOOL_REGISTRY: dict[str, RegisteredTool] = {}


def register_tool[**P, R](
    name: str,
) -> Callable[[Callable[P, Awaitable[R]]], Callable[P, Awaitable[R]]]:
    """最小异步注册器：只保存当前收到的对象，再原样返回。"""

    # 外层调用先捕获 name，返回真正接收 function 的 decorate；这是带参数装饰器的两阶段结构。
    def decorate(function: Callable[P, Awaitable[R]]) -> Callable[P, Awaitable[R]]:
        # 显式拒绝同名覆盖，避免后定义的函数悄悄替换已经注册的工具。
        if name in TOOL_REGISTRY:
            raise ValueError(f"工具名称已注册: {name}")

        # cast 只告诉静态检查器在异构注册表边界按宽类型看待 function，运行时不会包装或转换对象。
        TOOL_REGISTRY[name] = RegisteredTool(name, cast(AnyAsyncTool, function))

        # 原样返回同一个对象，所以注册器本身不改变函数身份；外层装饰器的顺序因此格外重要。
        return function

    return decorate


async def invoke_registered_tool(
    name: str,
    /,
    *args: object,
    **kwargs: object,
```

```python

async def invoke_registered_tool(
    name: str,
    /,
    *args: object,
    **kwargs: object,
) -> object:
    # 签名中的 `/` 表示 name 只能按位置传；`*args` 收集额外位置参数为 tuple，
    # `**kwargs` 收集关键字参数为 dict，再原样转发给运行时选中的异构工具。
    try:
        metadata: RegisteredTool = TOOL_REGISTRY[name]
    except KeyError:
        # `from None`隐藏底层 KeyError 的异常链，把动态入口对外统一为更明确的 LookupError。
        raise LookupError(f"未注册工具：{name}") from None

    # function(...) 先产生 Awaitable，await 后才得到动态结果；返回类型只能静态标为 object。
    return await metadata.function(*args, **kwargs)


# 装饰器从下往上应用：先用 observe_async 得到观测 wrapper，再由 register_tool 保存 wrapper。
# 因此通过公开名字直接调用和通过字符串注册表调用，都会经过相同的日志/计时层。
@register_tool("lookup")
@observe_async
async def lookup_documents(
    query: str,
    gate: ToolGate,
    /,
    *,
    limit: int = 2,
) -> ToolObservation:
    """在可控本地替身中模拟一次文档工具 I/O。"""

    # `/` 左侧的 query、gate 只能按位置传，`*` 右侧的 limit 只能按关键字传。
    # 日志只记录长度、上限和状态，不把 query、请求正文或凭据原样写入日志。
    LOGGER.info("文档查询等待中 query_length=%d limit=%d", len(query), limit)

    # 先通知请求 Task“工具已到达等待点”，再等它统一放行，模拟可控的异步 I/O
```

```
703  async def lookup_documents(
715
716      # 先通知请求 Task“工具已到达等待点”，再等它统一放行，模拟可控的异步 I/O。
717      gate.started.set()
718      await gate.may_finish.wait()
719      LOGGER.info("文档查询完成 result_count=%d", min(limit, 2))
720
721      # 工具在自己的 Task Context 中读取请求/运行标识，形成之后可断言的观察结果。
722      return ToolObservation(query, REQUEST_ID.get(), RUN_ID.get())
723
724
725  class ToolExecutionError(RuntimeError):
726      """本地替身的受控失败。"""
727
728
729  @register_tool("fail")
730  @observe_async
731  async def failing_tool(gate: ToolGate) -> None:
732      # 先停在 Event 上，让外层确定失败发生的时点；放行后抛受控异常，验证 wrapper 的 error 路径。
733      gate.started.set()
734      await gate.may_finish.wait()
735      raise ToolExecutionError("受控工具失败")
736
737
738  @register_tool("wait")
739  @observe_async
740  async def waiting_tool(gate: ToolGate) -> str:
741      # 取消实验不会设置 may_finish；Task 正在这个 await 暂停时会收到 CancelledError。
742      gate.started.set()
743      await gate.may_finish.wait()
744      return "不应在取消示例中成功"
745
746
747  # 这里故意反转顺序：register_tool 先保存原函数，observe_async 后来才包装公开名字。
748  # 因而从注册表调用会绕过观测层，这不是推荐写法，而是用来证明装饰器组合语义。
749  @observe_async
750  @register_tool("registered-before-observed")
```

```
746
747 # 这里故意反转顺序：register_tool 先保存原函数，observe_async 后来才包装公开名字。
748 # 因而从注册表调用会绕过观测层，这不是推荐写法，而是用来证明装饰器组合语义。
749 @observe_async
750 @register_tool("registered-before-observed")
751 async def registered_before_observed() -> str:
752     """只用于观察反向顺序：注册表会保存未包装的原函数。"""
753
754     return "order probe"
755
756
757 async def demonstrate_async_wrapper() -> None:
758     print_section("6. async wrapper 的计时覆盖真正 await，wraps 保留元信息")
759     started = asyncio.Event()
760     may_finish = asyncio.Event()
761
762     # 调用同步反例 wrapper 已经打印计时并返回 Awaitable，但 gated_coroutine 函数体尚未运行。
763     awaitable: Awaitable[str] = gated_coroutine(started, may_finish)
764     print("  同步 wrapper 返回时，原函数体已经开始:", started.is_set())
765     assert not started.is_set()
766
767     # ensure_future 把 Awaitable 安排进事件循环；对这里的 coroutine 来说效果相当于创建 Task。
768     task: Task[str] = asyncio.ensure_future(awaitable)
769
770     # started 被设置说明原函数体已开始；may_finish 未放行，所以 task.done() 仍应为 False。
771     await started.wait()
772     print("  调度后函数体开始、但尚未完成:", started.is_set() and not task.done())
773     may_finish.set()
774     assert await task == "真正的异步工作已完成"
775
776     # wraps 使函数名和 inspect.signature 默认沿 __wrapped__ 链显示原始签名。
777     print("  装饰后的名字:", lookup_documents.__name__)
778     print("  inspect.signature:", inspect.signature(lookup_documents))
779     print("  lookup 注册的是最终 wrapper:", TOOL_REGISTRY["lookup"].function is lookup_documents)
780     print(
781         "  反向顺序注册的是原函数:",
782         TOOL_REGISTRY["registered-before-observed"].function
```

```
async def demonstrate_async_wrapper() -> None:
    print(
        "  反向顺序注册的是原函数:",
        TOOL_REGISTRY["registered-before-observed"].function
        is inspect.unwrap(registered_before_observed),
    )

    # `is` 核对注册表保存的函数对象身份：推荐顺序保存 wrapper，反向顺序保存 unwrap 后的原函数。
    assert lookup_documents.__name__ == "lookup_documents"
    assert TOOL_REGISTRY["lookup"].function is lookup_documents
    assert TOOL_REGISTRY["registered-before-observed"].function is inspect.unwrap(
        registered_before_observed
    )


# ---------------------------------------------------------------------------
# 7. 两个请求、多个工具：正确日志归属
# ---------------------------------------------------------------------------
async def run_observed_request(
    request_id: str,
    run_id: str,
    queries: tuple[str, ...],
) -> tuple[ToolObservation, ...]:
    # 请求入口先在当前请求 Task 的 Context 建立两个关联字段；之后创建的工具 Task 会复制它们。
    # 每次 set 都返回独立 Token，finally 必须按与设置相反的顺序恢复。
    request_token: Token[str] = REQUEST_ID.set(request_id)
    run_token: Token[str] = RUN_ID.set(run_id)
    try:
        # len(queries) 是有界计数；原始 query 不进入请求级日志。
        LOGGER.info("请求开始 tool_count=%d", len(queries))

        # 每个 query 对应一个独立 ToolGate。生成器表达式逐项 create，再由 tuple 固定保存。
        gates: tuple[ToolGate, ...] = tuple(ToolGate.create() for _ in queries)

        # zip(..., strict=True) 成对提供 query 和 gate；长度不一致会抛 ValueError，而不是静默截断。
        # enumerate 从 1 产生稳定序号。create_task 的发生点已经处于正确请求 Context 中，
```

```python
async def run_observed_request(
        # zip(..., strict=True) 成对提供 query 和 gate；长度不一致会抛 ValueError，而不是静默截断。
        # enumerate 从 1 产生稳定序号。create_task 的发生点已经处于正确请求 Context 中，
        # 所以每个 invoke_registered_tool Task 都继承本请求的 request_id 和 run_id。
        tasks: tuple[Task[object], ...] = tuple(
            asyncio.create_task(
                invoke_registered_tool("lookup", query, gate, limit=2),
                name=f"context:{request_id}:lookup:{index}",
            )
            for index, (query, gate) in enumerate(zip(queries, gates, strict=True), start=1)
        )

        # 先等待所有工具进入 I/O 等待点，再逐一放行，避免日志条数和状态断言依赖机器速度。
        await asyncio.gather(*(gate.started.wait() for gate in gates))
        for gate in gates:
            gate.may_finish.set()

        # 动态注册入口的静态返回类型是 `object；运行时结果仍按 tasks 的输入顺序组成 list。
        dynamic_results: list[object] = await asyncio.gather(*tasks)
        results: list[ToolObservation] = []
        for result in dynamic_results:
            # isinstance 在动态边界收窄类型，也防止注册表内容与 lookup 合同不一致时悄悄返回脏数据。
            if not isinstance(result, ToolObservation):
                raise TypeError("lookup 注册项返回了意外类型")
            results.append(result)
        LOGGER.info("请求成功 success_count=%d", len(results))
        return tuple(results)
    finally:
        # 后设置的 RUN_ID 先 reset，再恢复 REQUEST_ID，使嵌套生命周期像栈一样退出。
        # 工具失败、请求取消或正常 return 都会经过这里。
        RUN_ID.reset(run_token)
        REQUEST_ID.reset(request_token)


async def demonstrate_concurrent_requests() -> None:
    print_section("7. 正例：两个请求各派生多个工具 Task，日志不串请求")
```

```python
async def demonstrate_concurrent_requests() -> None:
    print_section("7. 正例：两个请求各派生多个工具 Task，日志不串请求")

    # 父级示例 Task 当前仍是默认 Context。两个请求 Task 各自在自身函数入口 set 不同值，
    # 再分别派生两个工具 Task，形成两棵互相隔离的 Context 继承树。
    request_a: Task[tuple[ToolObservation, ...]] = asyncio.create_task(
        run_observed_request("REQ-100", "RUN-ALPHA", ("alpha", "beta")),
        name="request:REQ-100",
    )
    request_b: Task[tuple[ToolObservation, ...]] = asyncio.create_task(
        run_observed_request("REQ-200", "RUN-BETA", ("gamma", "delta")),
        name="request:REQ-200",
    )

    # 完成顺序可交错，但 gather 返回值仍按 request_a、request_b 的输入顺序解包。
    results_a: tuple[ToolObservation, ...], results_b: tuple[ToolOb… = await asyncio.gather(request_a, request_b)

    # 集合推导式提取每组结果中出现过的 ID；只得到一个元素证明组内两个工具归属一致。
    assert {item.observed_request_id for item in results_a} == {"REQ-100"}
    assert {item.observed_run_id for item in results_a} == {"RUN-ALPHA"}
    assert {item.observed_request_id for item in results_b} == {"REQ-200"}
    assert {item.observed_run_id for item in results_b} == {"RUN-BETA"}

    # 列表推导式从捕获的所有 LogRecord 中只选工具 Task 日志。
    # getattr(..., "") 提供安全回退值，避免没有 task_name 的记录导致 AttributeError。
    tool_records: list[LogRecord] = [
        record
        for record in CAPTURED_RECORDS
        if str(getattr(record, "task_name", "")).startswith("context:")
    ]

    # Task 名按 `context:<request_id>:lookup:<index>` 编码；split 最多切三次，索引 1 是预期请求。
    # 这里逐条比较日志 Filter 注入的 request_id，并核对 wrapper 注入的 tool_name。
    for record in tool_records:
        task_name = str(record.__dict__["task_name"])
        expected_request: str = task_name.split(":", maxsplit=3)[1]
```

```
846  async def demonstrate_concurrent_requests() -> None:
876
877      # Task 名按 `context:<request_id>:lookup:<index>` 编码；split 最多切三次，索引 1 是预期请求。
878      # 这里逐条比较日志 Filter 注入的 request_id，并核对 wrapper 注入的 tool_name。
879      for record in tool_records:
880          task_name = str(record.__dict__["task_name"])
881          expected_request: str = task_name.split(":", maxsplit=3)[1]
882          assert record.__dict__["request_id"] == expected_request
883          assert record.__dict__["tool_name"] == "lookup_documents"
884      print("  已按 task 名逐条核对工具日志数:", len(tool_records))
885      print("  入口退出后的默认 request_id:", REQUEST_ID.get())
886
887      # 每个工具产生开始、等待、完成、成功四条日志：2 个请求 * 2 个工具 * 4 = 16。
888      # 最后一条验证请求和工具的 finally 已把三层 ContextVar 全部恢复默认值。
889      assert len(tool_records) == 16
890      assert REQUEST_ID.get() == RUN_ID.get() == TOOL_NAME.get() == "-"
891
892
893  # ---------------------------------------------------------------------------
894  # 8. 异常与取消继续传播，且不能记成成功
895  # ---------------------------------------------------------------------------
896  # 失败请求与正常请求采用同样的上下文入口，只是派生的工具最终抛 ToolExecutionError。
897  async def request_that_fails() -> None:
898      request_token: Token[str] = REQUEST_ID.set("REQ-FAIL")
899      run_token: Token[str] = RUN_ID.set("RUN-FAIL")
900      try:
901          gate: ToolGate = ToolGate.create()
902
903          # Task 创建时复制失败请求的 Context；Event 保证异常要等外层明确放行后才发生。
904          task: Task[object] = asyncio.create_task(
905              invoke_registered_tool("fail", gate),
906              name="failure:controlled-tool",
907          )
908          await gate.started.wait()
909          gate.may_finish.set()
910
```

```python
async def request_that_fails() -> None:
        # await task 不捕获异常，因此工具的原 ToolExecutionError 会继续离开本函数。
        await task
    finally:
        # 即使上一行抛异常，这两个 reset 仍执行，避免失败请求污染后续示例。
        RUN_ID.reset(run_token)
        REQUEST_ID.reset(request_token)


async def demonstrate_failure_propagation() -> None:
    print_section("8. 普通异常：记录 traceback 后仍把原异常交给调用方")

    # 保存切片起点，之后只检查这次失败新增的记录，不会混入前七节的日志。
    first_record: int = len(CAPTURED_RECORDS)
    try:
        await request_that_fails()
    except ToolExecutionError as error:
        # 能在这里捕获同一异常类型，说明观测 wrapper 记录后使用了裸 raise，没有改写业务合同。
        print("  调用方收到原异常:", type(error).__name__)
    else:
        # try 的 else 只在没有异常时执行；走到这里反而表示示例失败，因此主动抛断言错误。
        raise AssertionError("受控工具失败应传播 ToolExecutionError")

    # `records[start:]` 是从起点到末尾的切片，会创建一个浅的新 list；LogRecord 对象本身不复制。
    new_records: list[LogRecord] = CAPTURED_RECORDS[first_record:]

    # getMessage() 执行 logging 的消息模板参数替换，便于在最终正文中查终态标记。
    messages: list[str] = [record.getMessage() for record in new_records]

    # any 在遇到第一个 True 时短路：本次必须有 error，且绝不能出现 success。
    assert any("status=error" in message for message in messages)
    assert not any("status=success" in message for message in messages)
    assert REQUEST_ID.get() == RUN_ID.get() == TOOL_NAME.get() == "-"


# 取消请求在入口和工具 wrapper 两层都记录 cancelled，但两层都必须继续传播同一个取消。
```

```python
# 取消请求在入口和工具 wrapper 两层都记录 cancelled，但两层都必须继续传播同一个取消。
async def cancellable_request(gate: ToolGate) -> str:
    request_token: Token[str] = REQUEST_ID.set("REQ-CANCEL")
    run_token: Token[str] = RUN_ID.set("RUN-CANCEL")
    try:
        LOGGER.info("可取消请求开始")
        try:
            result: object = await invoke_registered_tool("wait", gate)
        except asyncio.CancelledError:
            # 入口只增加请求级取消日志；裸 raise 让最外层仍能感知 Task 已取消。
            LOGGER.warning("请求入口继续传播取消 status=cancelled")
            raise

        # 动态注册入口返回 object；cast 只向类型检查器收窄为 str，不做运行时转换。
        return cast(str, result)
    finally:
        # CancelledError 也会执行 finally，所以取消不是跳过资源/上下文清理的捷径。
        RUN_ID.reset(run_token)
        REQUEST_ID.reset(request_token)


async def demonstrate_cancellation_propagation() -> None:
    print_section("9. 用户取消：工具和入口都记录 cancelled，不记录 success")
    first_record: int = len(CAPTURED_RECORDS)
    gate: ToolGate = ToolGate.create()

    # 把整个请求放进独立 Task，外层才有一个明确可调用 cancel() 的所有者。
    request_task: Task[str] = asyncio.create_task(
        cancellable_request(gate),
        name="cancel:REQ-CANCEL",
    )

    # 等 waiting_tool 已进入 may_finish.wait() 再取消，确保验证的是运行中 await 的传播路径。
    await gate.started.wait()

    # cancel(message) 发出取消请求；它在 Task 下次获得执行机会时向协程注入 CancelledError。
```

```
966  async def demonstrate_cancellation_propagation() -> None:
979
980      # cancel(message) 发出取消请求；它在 Task 下次获得执行机会时向协程注入 CancelledError。
981      request_task.cancel("用户停止请求")
982      try:
983          # 只有 await 被取消的 Task，当前外层协程才真正观察到传播出来的 CancelledError。
984          await request_task
985      except asyncio.CancelledError:
986          print("  最外层收到 CancelledError，Task.cancelled():", request_task.cancelled())
987      else:
988          raise AssertionError("取消必须继续传播到最外层")
989
990      new_records: list[LogRecord] = CAPTURED_RECORDS[first_record:]
991      messages: list[str] = [record.getMessage() for record in new_records]
992
993      # sum(bool ...) 利用 bool 可按 0/1 计数：工具层和请求入口应各有一条 cancelled，共两条。
994      # success 必须为零，Task 最终状态也必须是 cancelled，而非普通“成功返回 None”。
995      assert sum("status=cancelled" in message for message in messages) == 2
996      assert not any("status=success" in message for message in messages)
997      assert request_task.cancelled()
998      assert REQUEST_ID.get() == RUN_ID.get() == TOOL_NAME.get() == "-"
999
1000
1001 # ---------------------------------------------------------------------------
1002 # 10. async generator 是另一种协议
1003 # ---------------------------------------------------------------------------
1004 # async generator 用 yield 逐项产出值；调用 token_stream() 得到异步迭代器，不是可直接 await 的
1005 # coroutine。`-> AsyncGenerator[str]` 描述产出 str 的流协议，不是“一次 await 得到最终 str”。
1006 async def token_stream() -> AsyncGenerator[str]:
1007     yield "token-1"
1008     yield "token-2"
1009
1010
1011 async def demonstrate_async_generator_boundary() -> None:
1012     print_section("10. 边界：普通协程 wrapper 不直接通用于 async generator")
1013
1014     # 此调用只创建 stream 对象；inspect 的两个检查分别针对“函数种类”和“对象是否可 await”。
```

```python
async def demonstrate_async_generator_boundary() -> None:
    print_section("10. 边界：普通协程 wrapper 不直接通用于 async generator")

    # 此调用只创建 stream 对象；inspect 的两个检查分别针对“函数种类”和“对象是否可 await”。
    stream: AsyncGenerator[str, None] = token_stream()
    print("  token_stream 是异步生成器函数:", inspect.isasyncgenfunction(token_stream))
    print("  调用结果可直接 await:", inspect.isawaitable(stream))

    # anext(stream) 异步取得下一项；async for 会重复做同类操作，直到生成器耗尽。
    first: str = await anext(stream)

    # 本例只消费第一项，因此显式 aclose() 关闭流，让生成器有机会执行其清理逻辑。
    await stream.aclose()
    print("  正确方式是 anext/async for，本次第一项:", first)
    assert first == "token-1"

    # 不可 await 正是普通 observe_async 不能直接包装 async generator 的协议原因。
    assert not inspect.isawaitable(stream)


async def run_examples() -> None:
    # 这是所有章节的异步编排入口：先配置一次 logger，再严格按编号串行执行各实验。
    configure_logging()

    # split()[0] 取版本字符串的第一个空白分隔片段；版本元组断言明确本章依赖 Python 3.14。
    print(f"Python 版本: {sys.version.split()[0]}")
    assert sys.version_info >= (3, 14)

    await demonstrate_global_leak()
    demonstrate_token_lifecycle()
    await demonstrate_task_context_copy()
    await demonstrate_shallow_context_copy()
    await demonstrate_to_thread_context()
    await demonstrate_async_wrapper()
    await demonstrate_concurrent_requests()
```

```python
async def run_examples() -> None:
    assert sys.version_info >= (3, 14)

    await demonstrate_global_leak()
    demonstrate_token_lifecycle()
    await demonstrate_task_context_copy()
    await demonstrate_shallow_context_copy()
    await demonstrate_to_thread_context()
    await demonstrate_async_wrapper()
    await demonstrate_concurrent_requests()
    await demonstrate_failure_propagation()
    await demonstrate_cancellation_propagation()
    await demonstrate_async_generator_boundary()

    print_section("完成")
    print("  所有受控实验执行完毕; ContextVar 已恢复默认值。")

    # 连续相等比较同时核对反例全局值和三个带默认值的 ContextVar，没有章节状态泄漏。
    assert GLOBAL_REQUEST_ID == REQUEST_ID.get() == RUN_ID.get() == TOOL_NAME.get() == "-"


# 同步 main 只负责建立并关闭事件循环；asyncio.run 会运行 run_examples coroutine 直至完成。
def main() -> None:
    asyncio.run(run_examples())


# 模块被直接执行时 __name__ 才是 "__main__"；被 import 时只完成定义，不自动运行全部实验。
if __name__ == "__main__":
    main()
```